MBLP131 · HAFTA 13

İşletim Sistemleri Temelleri

Hafta 13 · Süreçler ve İş Denetimi

Yazdığınız her komut bir sürece dönüşür. Bugün onu izlemeyi, duraklatmayı, taşımayı ve durdurmayı öğreniyorsunuz.

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 12. Hafta

Üç akış

Her süreç fd 0 (stdin), 1 (stdout) ve 2 (stderr) ile başlar.

Yönlendirme

> >> < 2> 2>&1 &> /dev/null — bunu kabuk, fork() ile exec() arasında yapar.

Borular

cmd1 | cmd2 iki süreci aynı anda çalıştırır; aralarında bir çekirdek tamponu vardır.

Süzgeçler

grep, sort, uniq -c, cut, wc, tr, head, tail: boru hatlarında birleşen küçük araçlar.

Geçen hafta kabuk her boru hattı için sessizce süreçler oluşturdu. Bugün bu süreçlere doğrudan bakıyoruz ve denetimlerini elimize alıyoruz.

Bugünün Gündemi

1. **Süreçlere bakmak** ps, pstree, top, /proc/<pid>/status — ve R S D T Z durum harfleri.
2. **İş denetimi** Ön plan ve arka plan, &, Ctrl+Z, Ctrl+C, jobs, fg, bg.
3. **Sinyaller** kill, kill -l, SIGTERM, SIGKILL ve SIGINT, SIGSTOP/SIGCONT, pkill, killall, nohup.
4. **Öncelik** nice ve renice: 5. haftanın nice değerleri, ölçülmüş olarak.
5. **Zombiler** Z durumunu görünür kılan küçük bir C programı.
6. **Laboratuvar** İşleri başlatmak, incelemek, askıya almak, sürdürmek ve sonlandırmak (puanlı).

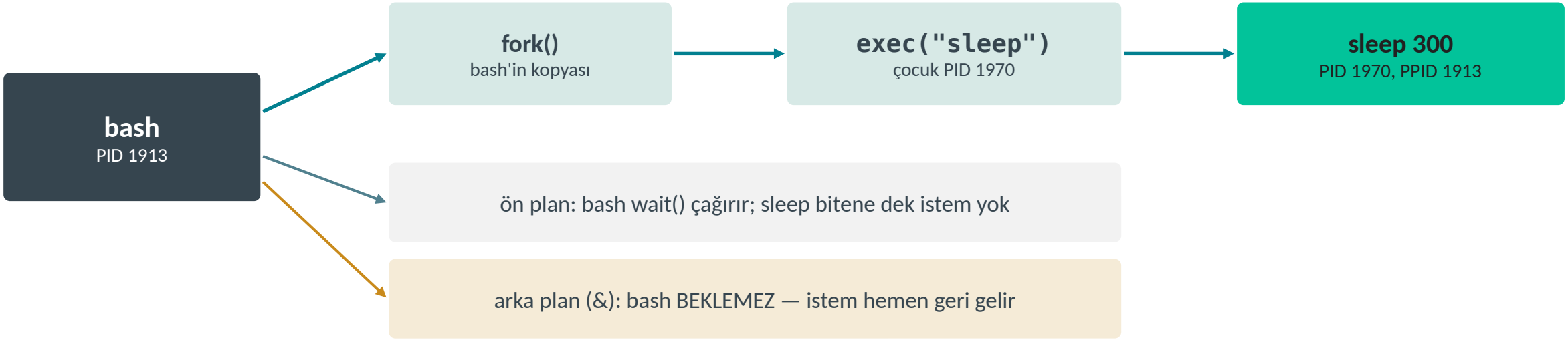
BÖLÜM 1

Süreçlere Bakmak

3. hafta kâğıt üzerindeydi — 13. hafta terminalde.

Her Komut Bir Süreçtir: fork, exec, wait

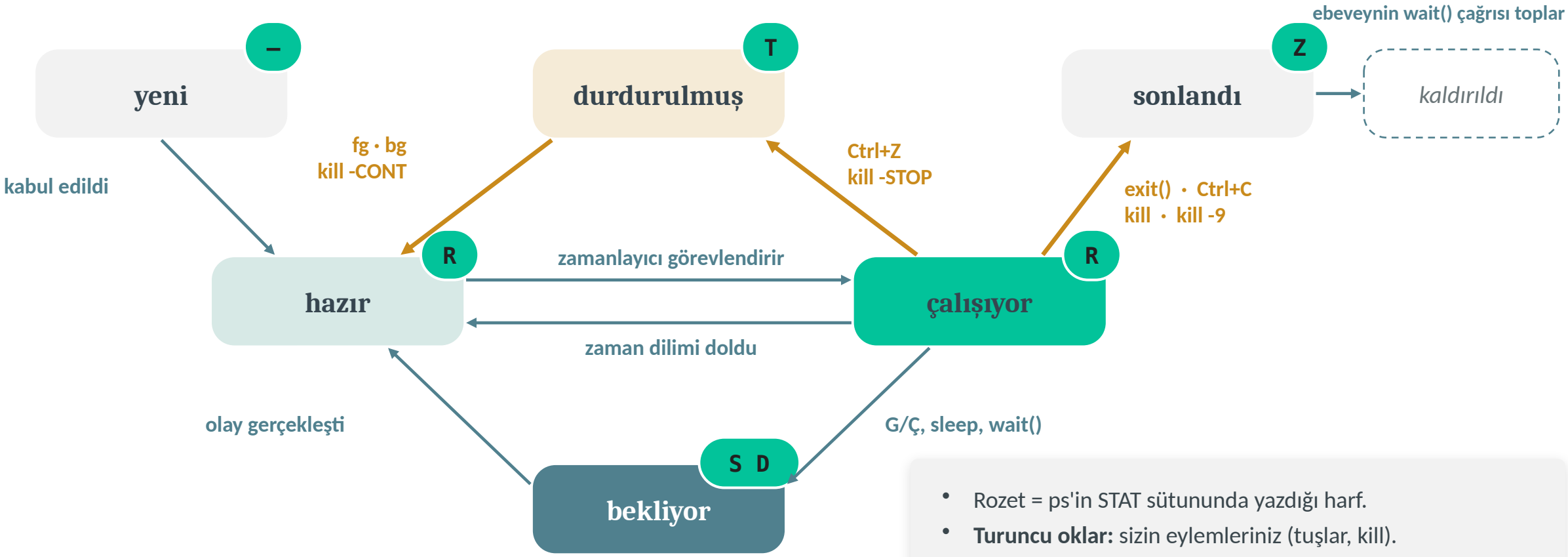
Süreç, yürütülmekte olan bir programdır; bir PID'si, bir ebeveyni (PPID) ve çekirdekte bir PCB'si vardır. Kabuk her komut için bir tane oluşturur.




```
student@okan-lab:~/lab$ sleep 300 &
[1] 1970
student@okan-lab:~/lab$ pstree -p $$
bash(1913)─┬─pstree(1975)
           ├─sleep(1970)
           └─yes(1971)
```

exec() öncesinde ve sonrasında PID aynıdır: çocuk numarasını korur ama yeni bir program çalıştırır. [1] iş numarası, 1970 PID'dir.

3. Hafta Durumları, ps STAT Harfleri ve Sizin Eylemleriniz



- Rozet = ps'in STAT sütununda yazdığı harf.
- **Turuncu oklar:** sizin eylemleriniz (tuşlar, kill).
- **Gri oklar:** çekirdek ve zamanlayıcı (3. hafta).

Linux durdurulmuş (T) durumunu ekler. R = hazır YA DA çalışıyor.

STAT Harfleri ve Ek Bayrakları

Harf	Anlamı	3. hafta durumu	Bugün nerede
R	çalışıyor ya da çalıştırılabilir (işlemcide ya da hazır kuyruğunda)	çalışıyor / hazır	yes > /dev/null
S	kesilebilir uyku: bir olayı bekliyor	bekliyor	sleep 300, bash
D	kesilemez uyku, genellikle disk G/Ç'si	bekliyor	(seyrek, kısa süreli)
T	Ctrl+Z ya da SIGSTOP ile durdurulmuş	— (Linux eki)	Ctrl+Z'den sonra
Z	zombi: bitti, henüz toplanmadı	sonlandı	./zombie (5. bölüm)

Harften sonraki ek bayraklar: s = oturum lideri (giriş kabuğunuz: Ss), + = ön plan grubunda (R+), N = düşük öncelik (nice uygulanmış: RN), < = yüksek öncelik, l = çok iş parçacıklı.

ps: Süreçlerinizin Anlık Görüntüsü

```
student@okan-lab:~/lab$ echo $$
1913
student@okan-lab:~/lab$ ps
  PID TTY          TIME CMD
 1913 pts/0    00:00:00 bash
 1969 pts/0    00:00:00 ps
student@okan-lab:~/lab$ sleep 300 &
[1] 1970
student@okan-lab:~/lab$ yes > /dev/null &
[2] 1971
student@okan-lab:~/lab$ ps
  PID TTY          TIME CMD
 1913 pts/0    00:00:00 bash
 1970 pts/0    00:00:00 sleep
 1971 pts/0    00:00:00 yes
 1972 pts/0    00:00:00 ps
student@okan-lab:~/lab$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   1913  1912  0 16:58 pts/0    00:00:00 -bash
student   1970  1913  0 16:58 pts/0    00:00:00 sleep 300
student   1971  1913 96 16:58 pts/0    00:00:00 yes
student   1973  1913  0 16:58 pts/0    00:00:00 ps -f
```

Neye dikkat etmeli?

- echo $$ kabuğun kendi PID'sini yazdırır: 1913.
- Yalın ps: yalnızca bu terminalin süreçleri (TTY pts/0).
- ps kendi listesinde yer alır — o da bir süreçtir.
- ps -f (tam biçim): UID, PPID, başlama zamanı ve komut satırının tamamı.
- sleep ve yes'in PPID'si = 1913: ebeveynleri kabuktur.

Sütunlarınızı Seçin, Ağacı Çizin

```
student@okan-lab:~/lab$ ps -o pid,ppid,stat,ni,%cpu,%mem,cmd
  PID  PPID STAT  NI %CPU %MEM CMD
 1913  1912 Ss     0  0.5  0.0 -bash
 1970  1913 S      0  0.0  0.0 sleep 300
 1971  1913 R      0 97.4  0.0 yes
 1974  1913 R+     0  0.0  0.0 ps -o pid,ppid,stat,ni,%cpu,%mem,cmd
student@okan-lab:~/lab$ pstree -p $$
bash(1913)─┬─pstree(1975)
           ├─sleep(1970)
           └─yes(1971)
```

```
student@okan-lab:~/lab$ kill %1 %2
student@okan-lab:~/lab$ jobs
[1]-  Terminated              sleep 300
[2]+  Terminated              yes > /dev/null
```

Neye dikkat etmeli?

- -o tam olarak istediğiniz sütunları listeler: burada durum (STAT), nice değeri (NI), %CPU ve %MEM.
- yes: R ve ≈ %97 işlemci. sleep: S ve %0.
- pstree -p, 3. haftanın ebeveyn–çocuk ağacını gerçek PID'lerle çizer.
- kill %1 %2 iki işi de bitirir; bash bir sonraki istemde "Terminated" bildirir.

Sütunlar Ne Anlama Gelir?

Sütun	Anlamı	Örnek
PID / PPID	süreç kimliği / ebeveynin süreç kimliği (3. hafta)	1970 / 1913
UID, USER	sürecin sahibi (11. hafta)	student
STAT	durum harfi + bayraklar (önceki slaytlar)	S, R+, Ss
%CPU	işlemci zamanı ÷ başlangıçtan beri geçen süre, yüzde	99.0 (yes)
%MEM	yerleşik bellek ÷ toplam RAM, yüzde	0.0
VSZ / RSS	sanal boyut / yerleşik küme (RAM'de), KiB (6–7. haftalar)	3132 / 1884
TTY	sürecin ait olduğu terminal; ? = yok (daemon)	pts/0
TIME	şimdiye dek kullanılan toplam işlemci zamanı	0:02
CMD, COMMAND	komut satırı	sleep 300

Çok iş parçacıklı bir süreçte %CPU 100'ü aşabilir: %200 = iki çekirdek tamamen dolu (4. hafta, iş parçacıkları).

Herkesin Süreçleri: ps -ef ve ps aux

```
student@okan-lab:~/lab$ ps -ef | head -4
UID          PID  PPID  C STIME TTY          TIME CMD
root           1     0  0 16:17 ?        00:00:02 /process_api --firecracker-init …
root           2     0  0 16:17 ?        00:00:00 [kthreadd]
root           3     2  0 16:17 ?        00:00:00 [pool_workqueue_release]
# ... ps -f -u student: yalnızca student kullanıcısı
student@okan-lab:~/lab$ ps -f -u student
UID          PID  PPID  C STIME TTY          TIME CMD
student     1913  1912  0 16:58 pts/0    00:00:00 -bash
student     1970  1913  0 16:58 pts/0    00:00:00 sleep 300
student     1971  1913 99 16:58 pts/0    00:00:01 yes
student     1978  1913  0 16:58 pts/0    00:00:00 ps -f -u student
student@okan-lab:~/lab$ ps aux | head -1
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
student@okan-lab:~/lab$ ps aux | grep '^student'
student     1913  0.6  0.0   8436  7532 pts/0    Ss   16:58   0:00 -bash
student     1970  0.0  0.0   3132  1996 pts/0    S    16:58   0:00 sleep 300
student     1971 99.0  0.0   3132  1884 pts/0    R    16:58   0:02 yes
student     1981  0.0  0.0   8340  4328 pts/0    R+   16:58   0:00 ps aux
student     1982  0.0  0.0   4100  2200 pts/0    S+   16:58   0:00 grep ^student
```

İki stil, aynı veri

- **-ef:** her süreç, tam biçim (System V stili).
- **aux:** her süreç; %CPU, %MEM, VSZ, RSS ile (BSD stili, tiresiz).
- **-u student:** yalnızca bir kullanıcı.
- PID 1 ilk süreçtir; çekirdek iş parçacıkları [köşeli parantez] içinde görünür.
- grep kendini bulur!

/proc/<pid>/status: PCB'yi Okumak

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2118
student@okan-lab:~/lab$ echo $!
2118
student@okan-lab:~/lab$ grep -E '^(Name|State|Pid|PPid|Uid|Threads|VmRSS)' …
Name:   sleep
State:  S (sleeping)
Pid:    2118
PPid:   2061
Uid:    1013    1013    1013    1013
VmRSS:      2004 kB
Threads:        1
student@okan-lab:~/lab$ grep ctxt /proc/$!/status
voluntary_ctxt_switches:        1
nonvoluntary_ctxt_switches:     2
student@okan-lab:~/lab$ cat /proc/$!/cmdline | tr '\0' ' '; echo
sleep 300
student@okan-lab:~/lab$ ls -l /proc/$!/cwd /proc/$!/exe
lrwxrwxrwx 1 student student 0 Oct 12 16:58 /proc/2118/cwd -> /home/student…
lrwxrwxrwx 1 student student 0 Oct 12 16:58 /proc/2118/exe -> /usr/bin/sleep
```

3. haftanın PCB alanları

- **State:** S (sleeping) = bekliyor.
- **Pid, PPid:** kimlik ve ebeveyn.
- **Uid:** sahip (11. hafta).
- **VmRSS:** bellek bilgisi.
- **ctxt_switches:** hesap bilgisi — voluntary = işlemciyi kendisi bıraktı, nonvoluntary = elinden alındı.
- **cwd, exe:** çalışma dizini ve program dosyası.

top: Birkaç Saniyede Bir Yenilenen Canlı Görünüm

```
student@okan-lab:~/lab$ top -b -n 1 -u student | head -15
top - 16:58:44 up 41 min,  0 user,  load average: 0.59, 0.34, 0.23
Tasks:  71 total,   2 running,  69 sleeping,   0 stopped,   0 zombie
%Cpu(s): 28.6 us, 23.8 sy,  0.0 ni, 42.9 id,  0.0 wa,  0.0 hi,  0.0 si,  4.8 st
MiB Mem :   8029.6 total,   7294.2 free,    598.8 used,    357.6 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   7430.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2260 student   20   0    3132   1876   1760 R 100.0   0.0   0:30.83 yes
 2204 student   20   0    8436   7528   3552 S   0.0   0.1   0:00.01 bash
 2261 student   20   0    3132   2004   1892 S   0.0   0.0   0:00.00 sleep
 2264 student   20   0    9380   5436   3308 R   0.0   0.1   0:00.00 top
 2265 student   20   0    3144   1956   1840 S   0.0   0.0   0:00.00 head
```

1. satır

çalışma süresi ve yük ortalaması: işlemci kullanan ya da işlemci bekleyen süreç sayısının 1, 5 ve 15 dakikalık ortalamaları.

2–3. satırlar

Durumlarına göre görevler (STAT'tan); BÜTÜN makinenin %Cpu(s) değeri: us = kullanıcı modu, sy = çekirdek modu (2. hafta), id = boşta.

Süreç listesi

PR = 20 + NI (5. hafta önceliği), S = durum, son aralıktaki %CPU: yes bir çekirdeğin %100'ünü kullanıyor.

ÇÖZÜMLÜ ÖRNEK

Yük Ortalamasını Okumak

Yük = çalışan ya da hazır (R) süreçlerin, artı disk bekleyen (D) süreçlerin ortalama sayısı. İşlemci çekirdeği sayısıyla karşılaştırın.

```
student@okan-lab:~/lab$ cat /proc/loadavg
0.59 0.34 0.23 3/98 2262
student@okan-lab:~/lab$ nproc
2
student@okan-lab:~/lab$ uptime
 16:58:45 up 41 min,  0 user,
           load average: 0.59, 0.34, 0.23
```

Bir yes bir çekirdek ister: top onun için %100 gösterir, ama 2 çekirdeğin ortalaması olan %Cpu(s) içinde yalnızca yaklaşık %50.

```
çekirdek c = 2  (nproc)

yük 0.59 → 0.59 / 2 = kapasitenin %29.5'i
yük 2.00 → 2.00 / 2 = %100: her çekirdek dolu
yük 3.00 → %150: ortalama 3 − 2 = 1 süreç
            hazır kuyruğunda bekler

# 1 dk > 15 dk: yük artıyor
# 1 dk < 15 dk: yük azalıyor
```

4. haftayla bağlantı: yük > çekirdek sayısı ise süreçler hazır kuyruğunda zaman geçirir — bekleme süreleri uzar.

KOPYA KÂĞIDI

Süreçleri İncelemek: Bir Bakışta

Komut	Ne gösterir?
ps · **ps -f**	bu terminalin süreçleri (kısa / tam biçim)
ps -ef · **ps aux**	bütün kullanıcıların bütün süreçleri
ps -u student	bir kullanıcının bütün süreçleri
ps -o pid,ppid,stat,ni,%cpu,cmd	tam olarak seçtiğiniz sütunlar
ps … --sort=-%cpu	sıralı, en yüksek işlemci kullanımı önce
pgrep -l name · **pgrep -c name**	eşleşen süreçlerin PID'leri (ve adları) / sayısı
pstree -p · **pstree -p $$**	PID'leriyle süreç ağacı
top · **htop**	canlı görünüm; q ile çıkılır
cat /proc/<pid>/status	metin olarak PCB
echo $$ · **echo $!**	bu kabuğun / son arka plan işinin PID'si

BÖLÜM 2

İş Denetimi

Bir terminal, birçok iş: ön plan, arka plan, durdurulmuş.

Ön Plan ve Arka Plan İşleri

Klavyeye yalnızca BİR iş sahip olur: ön plan işi. Ctrl+C ve Ctrl+Z ona iletilir.

Ön plan: komut

- Kabuk bekler (3. haftanın wait() çağrısı) — iş bitene dek istem yok.
- İş klavyenizi okur.
- Ctrl+C SIGINT gönderir: genellikle işi sonlandırır.
- Ctrl+Z SIGTSTP gönderir: işi durdurur (T durumu), istemi geri verir.
- Kısa komutlar ve etkileşimli her şey için (nano, less, top).

Arka plan: komut &

- Kabuk [iş] PID ve istemi hemen yazdırır.
- İş çalışırken siz çalışmaya devam edersiniz.
- Ctrl+C ona ulaşmaz: kill %n kullanın.
- Çıktısı yine ekranınıza gelir — yönlendirin (> dosya, 12. hafta).
- Uzun, etkileşimsiz işler için: derleme, kopyalama, indirme.

jobs BU kabuğun işlerini listeler. fg %n: ön plana getir. bg %n: durdurulmuş bir işi arka planda sürdür.

Ctrl+Z, jobs ve bg

```
student@okan-lab:~/lab$ sleep 100
^Z
[1]+  Stopped                 sleep 100
student@okan-lab:~/lab$ jobs
[1]+  Stopped                 sleep 100
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
 2345 Ss   -bash
 2401 T    sleep 100
 2402 R+   ps -o pid,stat,cmd
student@okan-lab:~/lab$ bg
[1]+ sleep 100 &
student@okan-lab:~/lab$ jobs -l
[1]+  2401 Running                 sleep 100 &
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
 2345 Ss   -bash
 2401 S    sleep 100
 2403 R+   ps -o pid,stat,cmd
```

Adım adım

- **sleep 100:** ön planda — istem yok.
- **Ctrl+Z (^Z):** durduruldu; istem geri geldi.
- **ps:** STAT T — durdurulmuş durumu.
- **bg:** arka planda sürdür; kabuk & ekler.
- **jobs -l:** artık Running, PID'siyle birlikte.
- **ps:** yeniden STAT S — sleep zamanlayıcısını bekliyor, takılmadı.

& ile Başlatmak, fg, Ctrl+C ve kill %n

```
student@okan-lab:~/lab$ sleep 200 &
[2] 2404
student@okan-lab:~/lab$ yes > /dev/null &
[3] 2405
student@okan-lab:~/lab$ jobs
[1]   Running                 sleep 100 &
[2]-  Running                 sleep 200 &
[3]+  Running                 yes > /dev/null &
student@okan-lab:~/lab$ fg %3
yes > /dev/null
^C
student@okan-lab:~/lab$ jobs
[1]-  Running                 sleep 100 &
[2]+  Running                 sleep 200 &
student@okan-lab:~/lab$ kill %1 %2
student@okan-lab:~/lab$ jobs
[1]-  Terminated              sleep 100
[2]+  Terminated              sleep 200
student@okan-lab:~/lab$ jobs
```

Adım adım

- **&:** [2] 2404 = 2 numaralı iş, PID 2404.
- **jobs:** + geçerli iş, - bir önceki.
- **fg %3:** yes ön plana gelir; kabuk komutunu yazdırır.
- **Ctrl+C (^C):** SIGINT onu bitirir.
- **kill %1 %2:** % iş numarası demektir.
- Terminated bir kez, bir sonraki istemden hemen önce bildirilir.

İşleri ve Süreçleri Adlandırmak

Yazdığınız	Anlamı	Örnek
%1, %2 ...	bu kabuğun n numaralı işi	fg %2, kill %1
%+ ya da %%	geçerli iş (+ ile işaretli); argümansız fg/bg de bunu kullanır	fg
%-	bir önceki iş (- ile işaretli)	bg %-
%sleep	komutu sleep ile başlayan iş	kill %sleep
1970	bir PID — her terminalde, her araçla geçerli	kill 1970
$$	geçerli kabuğun PID'si	echo $$ → 1913
$!	en son arka plan işinin PID'si	kill $!

İş numaraları (%n) yalnızca işleri başlatan kabuğun içinde vardır. PID'ler geneldir: başka bir terminaldeki ps, top ve kill de onları görür.

ARA

10 Dakika Mola

Sırada: sinyaller — kill gerçekte nasıl çalışır — ardından öncelikler ve zombiler.

BÖLÜM 3

Sinyaller

Çekirdeğin bir sürece ilettiği küçük, numaralı iletiler.

Sinyal, Bir Süreç İçin Yazılımsal Bir Kesmedir

2. hafta: donanım kesmesi işlemciyi durdurur ve bir işleyici çalıştırır. Sinyal aynısını tek bir süreç için yapar.



Süreç sinyalle ne yapar?

- **Varsayılan eylem:** sonlan, dur, sürdür ya da yok say — her sinyal için sabittir.
- **Yakala:** kendi işleyici fonksiyonunu çalıştır (örneğin işi kaydet, sonra çık).
- **Yok say:** sinyalin hiçbir etkisi olmaz.
- **İstisnalar:** SIGKILL (9) ve SIGSTOP (19) yakalanamaz ve yok sayılamaz.

Kurallar

- Bir kullanıcı yalnızca KENDİ süreçlerine sinyal gönderebilir (root: herkesinkine).
- kill her zaman öldürmez: bir sinyal gönderir. Varsayılan sinyal 15'tir, SIGTERM.
- Sinyallerin adları ve numaraları vardır: kill -l.

Bilmeniz Gereken Sinyaller

No.	Ad	Gönderen	Varsayılan eylem	Yakalanır mı?
1	SIGHUP	terminali kapatmak ("hang up")	sonlan	evet
2	SIGINT	Ctrl+C	sonlan	evet
9	SIGKILL	kill -9 — son çare	hemen sonlan	HAYIR
15	SIGTERM	kill (varsayılan)	sonlan	evet
17	SIGCHLD	çekirdek, bir çocuk bittiğinde	yok say	evet
18	SIGCONT	fg, bg, kill -CONT	durmuşsa sürdür	evet
19	SIGSTOP	kill -STOP	dur	HAYIR
20	SIGTSTP	Ctrl+Z	dur	evet

Ctrl+\ SIGQUIT (3) gönderir: SIGINT gibidir ama bir bellek dökümü (core dump) da yazabilir. SIGSEGV (11), çekirdeğin geçersiz bir bellek erişimine yanıtıdır (7. hafta).

kill -l, SIGTERM, SIGKILL, SIGINT — ve Çıkış Durumu

```
student@okan-lab:~/lab$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
# ... toplam 64 sinyal
student@okan-lab:~/lab$ sleep 300 &
[1] 2541
student@okan-lab:~/lab$ kill $!
student@okan-lab:~/lab$ wait $!
[1]+  Terminated              sleep 300
student@okan-lab:~/lab$ echo $?
143
student@okan-lab:~/lab$ sleep 300 &
[1] 2542
student@okan-lab:~/lab$ kill -9 $!
student@okan-lab:~/lab$ wait $!
[1]+  Killed                  sleep 300
student@okan-lab:~/lab$ echo $?
137
student@okan-lab:~/lab$ sleep 300 &
[1] 2543
student@okan-lab:~/lab$ kill -INT $!
student@okan-lab:~/lab$ wait $!
[1]+  Interrupt               sleep 300
student@okan-lab:~/lab$ echo $?
130
```

Neye dikkat etmeli?

- kill -l bütün sinyalleri listeler; kill -l 15 TERM, kill -l KILL 9 yazdırır.
- wait $! işi bekler ve nasıl bittiğini bildirir: Terminated, Killed, Interrupt.
- $? = son komutun çıkış durumu.

```
n sinyaliyle öldü → $? = 128 + n
SIGTERM: 128 + 15 = 143
SIGKILL: 128 +  9 = 137
SIGINT : 128 +  2 = 130
```

Duraklat ve Sürdür; Süreçlere Adlarıyla Sinyal Gönder

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2544
student@okan-lab:~/lab$ kill -STOP $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
 2544 T    sleep 300

[1]+  Stopped                 sleep 300
student@okan-lab:~/lab$ kill -CONT $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
 2544 S    sleep 300
student@okan-lab:~/lab$ kill $!
```

```
student@okan-lab:~/lab$ sleep 401 &
[2] 2548
[1]   Terminated              sleep 300
student@okan-lab:~/lab$ sleep 402 &
[3] 2549
student@okan-lab:~/lab$ yes > /dev/null &
[4] 2550
student@okan-lab:~/lab$ pgrep -u student -l sleep
2548 sleep
2549 sleep
student@okan-lab:~/lab$ pkill -u student sleep
[2]   Terminated              sleep 401
[3]-  Terminated              sleep 402
student@okan-lab:~/lab$ killall -u student yes
[4]+  Terminated              yes > /dev/null
```

pgrep / pkill / killall süreçleri ADLARINA göre seçer. Her zaman -u student ekleyin: yoksa başka kullanıcıların süreçlerini de denerler.

Sinyalleri Yakalayan Bir Program: catch.c

```
static volatile sig_atomic_t got = 0;
static void handler(int sig) { got = sig; }  /* minicik! */

sa.sa_handler = handler;
sigaction(SIGINT,  &sa, NULL);   /* Ctrl+C, 2: yakalanır */
sigaction(SIGTERM, &sa, NULL);   /* kill,  15: yakalanır */
if (sigaction(SIGKILL, &sa, NULL) < 0)  /* 9: reddedilir */
    perror("catch: sigaction(SIGKILL)");

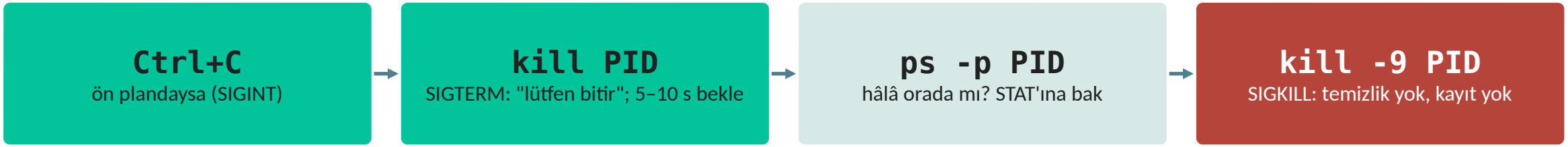
for (;;) {
    pause();                     /* sinyal gelene dek uyu */
    if (got == SIGINT)
        printf("... ignoring it, still running\n");
    if (got == SIGTERM) {
        printf("... cleaning up, bye\n");
        return 0;
    }
}
```

SIGINT yakalanır (yalnızca bir ileti); SIGTERM temiz bir çıkış sağlar (Done); SIGKILL işleyicisi reddedilir ve kill -9 her zaman işe yarar.

```
student@okan-lab:~/lab/src$ ./catch
catch: sigaction(SIGKILL): Invalid argument
catch: PID 2694, waiting for signals
^Ccatch: got SIGINT (2) -- ignoring it, still running
^Ccatch: got SIGINT (2) -- ignoring it, still running
^Z
[1]+  Stopped                 ./catch
student@okan-lab:~/lab/src$ bg
[1]+ ./catch &
student@okan-lab:~/lab/src$ kill -INT %1
student@okan-lab:~/lab/src$ catch: got SIGINT (2) -- ignoring it, still …
kill -TERM %1
student@okan-lab:~/lab/src$ catch: got SIGTERM (15) -- cleaning up, bye
jobs
[1]+  Done                    ./catch
student@okan-lab:~/lab/src$ ./catch &
[1] 2695
student@okan-lab:~/lab/src$ catch: sigaction(SIGKILL): Invalid argument
catch: PID 2695, waiting for signals
kill -9 %1
student@okan-lab:~/lab/src$ jobs
[1]+  Killed                  ./catch
```

İYİ UYGULAMA

Bir Süreci Güvenle Durdurmak: Önce Rica Et, En Son Zorla



Neden her zaman kill -9 değil?

- Program açık belgeleri kaydedemez, tamponlarını boşaltamaz.
- Geçici dosyalar ve kilit dosyaları geride kalır; bir veritabanı onarım isteyebilir.
- Çocuklarına haber verilmez; yetim kalırlar.

kill -9'un İŞE YARAMADIĞI durumlar

- **Z (zombi):** zaten ölü — onu yalnızca ebeveynin wait() çağrısı kaldırır.
- **D (disk bekleme):** sinyal G/Ç bittiğinde iletilir.
- **Başkasının süreci:** Operation not permitted (root'a başvurun).

Kapanan Bir Terminalden Sağ Çıkmak: nohup

```
student@okan-lab:~/lab$ echo "my shell is PID $$"
my shell is PID 2775
student@okan-lab:~/lab$ sleep 500 &
[1] 2831
student@okan-lab:~/lab$ echo "last background job is PID $!"
last background job is PID 2831
student@okan-lab:~/lab$ nohup sleep 600 &
[2] 2832
student@okan-lab:~/lab$ nohup: ignoring input and appending output to 'nohup.ou…
nohup sleep 700 > /dev/null 2>&1 &
[3] 2834
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  nohup.out  photos  projects  scripts  src
student@okan-lab:~/lab$ jobs -l
[1]   2831 Running                 sleep 500 &
[2]-  2832 Running                 nohup sleep 600 &
[3]+  2834 Running                 nohup sleep 700 > /dev/null 2>&1 &
student@okan-lab:~/lab$ kill -HUP %1 %2 %3
student@okan-lab:~/lab$ jobs -l
[1]   2831 Hangup                  sleep 500
[2]-  2832 Running                 nohup sleep 600 &
[3]+  2834 Running                 nohup sleep 700 > /dev/null 2>&1 &
student@okan-lab:~/lab$ kill %2 %3
```

Neye dikkat etmeli?

- $$ = kabuk, $! = son arka plan işi.
- Terminali kapatmak işlerine SIGHUP (1) gönderir; bunu kill -HUP ile taklit ediyoruz.
- Yalın sleep 500: Hangup — gitti.
- nohup işleri SIGHUP'ı yok sayar: hâlâ Running.
- Yönlendirmezseniz nohup'ın çıktısı nohup.out dosyasına gider.

BÖLÜM 4 VE 5

Öncelikler ve Zombiler

5. haftanın zamanlaması ve 3. haftanın sonlandırması, ölçülmüş olarak.

nice Değerleri: Öteki Süreçlere Nazik Davranmak



5. hafta tekrarı

- Küçük sayı = yüksek öncelik, 5. haftadaki gibi.
- Linux işlemciyi ağırlığa göre paylaştırır: nice 0 → 1024, nice 10 → 110; her adım ≈ × 1,25.
- top PR = 20 + NI gösterir.

```
# TEK çekirdekte işlemci yoğun iki iş
pay(nice 0)  = 1024 / (1024 + 110)
             = %90.3
pay(nice 10) =  110 / (1024 + 110)
             =  %9.7
oran         = 1024 / 110 = 9.31
denetim: 1.25^10 = 9.31
```

Uygulamada nice ve renice

```
student@okan-lab:~/lab$ nice
0
student@okan-lab:~/lab$ nice -n 10 yes > /dev/null &
[1] 2971
student@okan-lab:~/lab$ ps -o pid,ni,stat,cmd
  PID  NI STAT CMD
 2914   0 Ss   -bash
 2971  10 RN   yes
 2972   0 R+   ps -o pid,ni,stat,cmd
student@okan-lab:~/lab$ renice -n 15 -p $!
2971 (process ID) old priority 10, new priority 15
student@okan-lab:~/lab$ renice -n 5 -p $!
renice: failed to set priority for 2971 (process ID): Permission denied
student@okan-lab:~/lab$ nice -n -5 sleep 1
nice: cannot set niceness: Permission denied
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ taskset -c 1 yes > /dev/null &
[2] 2976
[1]   Terminated              nice -n 10 yes > /dev/null
student@okan-lab:~/lab$ taskset -c 1 nice -n 10 yes > /dev/null &
[3] 2977
student@okan-lab:~/lab$ ps -o pid,ni,stat,%cpu,cmd
  PID  NI STAT %CPU CMD
 2914   0 Ss    0.0 -bash
 2976   0 R    89.3 yes
 2977  10 RN    9.6 yes
 2978   0 R+    0.0 ps -o pid,ni,stat,%cpu,cmd
student@okan-lab:~/lab$ kill %2 %3
```

Neye dikkat etmeli?

- Tek başına nice geçerli değeri yazdırır: 0.
- nice -n 10: NI 10, STAT RN (N = nice uygulanmış).
- renice yukarı (10 → 15): izinli. Aşağı (15 → 5): Permission denied.
- Tek çekirdekte iki yes (taskset -c 1): %89,3 ve %9,6, oran 9,30 — öngörülen 9,31.

zombie.c: Kimsenin Beklemediği Bir Çocuk

```
pid_t pid = fork();                /* bir süreç ikiye ayrılır */

if (pid == 0) {                    /* ---- çocuk ---- */
    printf("child  %d: exiting now with status 7\n", getpid());
    exit(7);                       /* sonlandı -> toplanana dek zombi */
}
/* ---- ebeveyn: secs saniye boyunca wait() ÇAĞIRMAZ ---- */
printf("parent %d: child is %d, sleeping %d s without wait()\n",
       getpid(), pid, secs);
sleep(secs);                       /* bu sırada çocuk bir zombidir */

waitpid(pid, &status, 0);          /* çıkış durumunu topla (reap) */
printf("parent %d: reaped child %d, exit status %d\n",
       getpid(), pid, WEXITSTATUS(status));
```

Plan

- fork(): ebeveyn ve çocuk (3. hafta).
- Çocuk hemen 7 durumuyla çıkar.
- Ebeveyn wait() ÇAĞIRMADAN 25 s uyur: bu sırada çocuk bir zombidir.
- Sonra waitpid() 7 durumunu toplar: zombi kaldırılır.

Z Durumunu Görmek — ve Onu Öldürememek

```
student@okan-lab:~/lab/src$ gcc -Wall -O2 -o zombie zombie.c
student@okan-lab:~/lab/src$ ./zombie 25 &
[1] 3119
student@okan-lab:~/lab/src$ parent 3119: child is 3120, sleeping 25 s without w…
child  3120: exiting now with status 7
ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3119  3057 S    ./zombie 25
 3120  3119 Z    [zombie] <defunct>
 3121  3057 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab/src$ grep State /proc/$(pgrep -u student -n zombie)/stat…
State:  Z (zombie)
student@okan-lab:~/lab/src$ kill -9 $(pgrep -u student -n zombie)
student@okan-lab:~/lab/src$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3119  3057 S    ./zombie 25
 3120  3119 Z    [zombie] <defunct>
 3125  3057 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab/src$ wait
parent 3119: reaped child 3120, exit status 7
[1]+  Done                    ./zombie 25
student@okan-lab:~/lab/src$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3128  3057 R+   ps -o pid,ppid,stat,cmd
```

Neye dikkat etmeli?

- PID 3120: STAT Z, [zombie] <defunct>, PPID 3119.
- /proc da aynısını söyler: State Z (zombie).
- kill -9 hiçbir şey yapmaz: süreç zaten ölü.
- Ebeveynin waitpid() çağrısından sonra: çıkış durumu 7 ve zombi gitti.

KOPYA KÂĞIDI

İşleri ve Süreçleri Denetlemek: Bir Bakışta

Komut / tuş	Etkisi
command &	arka planda başlat; [iş] PID yazdırır
Ctrl+C · Ctrl+Z	ön plan işine SIGINT · SIGTSTP: durdur
jobs · jobs -l	bu kabuğun işlerini listele (PID'lerle)
fg %n · bg %n	ön plana getir · arka planda sürdür
kill PID · kill %n	bir sürece / bir işe SIGTERM (15)
kill -9 · -STOP · -CONT	SIGKILL (son çare) · duraklat · sürdür
pkill -u student name · killall	süreçlere adlarıyla sinyal gönder
nohup command > log 2>&1 &	terminal kapandıktan sonra da çalışsın
nice -n 10 cmd · renice -n 15 -p PID	nice değeriyle başlat / değiştir (kullanıcı: yalnızca yukarı)

DERSTE LABORATUVAR · 30–40 DK

Laboratuvar Alıştırmaları (1/2): İnceleme ve Denetim

A · İnceleme (1–5)

- echo $$ ve onu ps içinde bulun
- ps -f: ps'in ebeveyni kim?
- sleep 1000 &: /proc/$!/status'tan State ve PPid
- iki sleep ile pstree -p $$
- R ve S: yes ve sleep yan yana

B · İş denetimi (6–9)

- sleep 500, Ctrl+Z, jobs, STAT T
- bg; jobs -l ve ps ile denetleyin
- üç iş; fg %2, sonra Ctrl+C
- kill %1 ve kill 1: açıklayın

~/lab içinde çalışın (önce bash mcmp117_lab_setup.sh). Her komutu ve çıktısını yazın: ders notlarındaki Laboratuvar Föyü 17 alıştırmanın tümünü ve puanlarını içerir.

DERSTE LABORATUVAR · 30–40 DK

Laboratuvar Alıştırmaları (2/2): Sinyal, Öncelik, Zombi

C · Sinyaller (10–13)

- SIGTERM, SIGKILL, SIGINT sonrası $?: 128 + n'yi açıklayın
- STOP ve CONT: STAT'ı izleyin
- pgrep -c, pkill -u student
- nohup ve yalın iş: ikisine de kill -HUP

D · Öncelik, zombiler, meydan okuma (14–17)

- nice -n 15, renice ile 19'a, sonra 0'a
- ./zombie 30 &: Z'yi bulun, kill -9 deneyin
- %CPU'ya göre ilk 3 süreç (--sort)
- bir 12. hafta boru hattıyla süreçlerinizi duruma göre sayın

Bitirince temizleyin: jobs boş olmalı ve pgrep -u student -l sleep hiçbir şey yazdırmamalı.

Hata İletileri ve Anlamları

Gördüğünüz	Neden?	Çözüm
-bash: kill: (1) - Operation not permitted	kill 1, PID 1 demektir (sistem!)	1. iş için kill %1
-bash: kill: %6: no such job	iş numaraları kabuğa özgüdür	önce jobs ile bakın
renice: ... Permission denied	kullanıcılar nice'ı düşüremez	yalnızca yükseltin (ya da root)
nice: cannot set niceness: Permission denied	kullanıcı olarak negatif nice	yine de nice 0 ile çalışır
istem geri gelmiyor	bir ön plan işi çalışıyor	Ctrl+C ya da Ctrl+Z, sonra bg
[1]+ Stopped ... (tty input)	arka plan işi klavyeyi istiyor	fg %1
Z <defunct> kill -9'dan sağ çıkıyor	zombiler zaten ölüdür	ebeveyni bitirin ya da düzeltin

Yeniden yazmadan önce iletiyi okuyun: bash, kill, renice ve nice hangi kuralı çiğnediğinizi size tam olarak söyler.

KISA SINAV

Kendinizi Sınayın

1. ps sürecinizin STAT değerini T gösteriyor. Buna en yakın 3. hafta durumu hangisi; onu hangi iki komut geri getirir?

2. Bir iş bitti ve echo $? 137 yazdırıyor. Ona ne oldu? 143 ne anlama gelirdi?

3. Tek çekirdekte işlemci yoğun iki sürecin nice değerleri 0 ve 10. Her biri yaklaşık hangi işlemci payını alır?

4. kill -9 bir zombiyi neden kaldıramaz? Onu ne kaldırır?

13. Hafta: Akılda Kalması Gerekenler

- **Süreçler:** her komut, kabuğun fork + exec ile oluşturduğu, PID'si ve PPID'si olan bir süreçtir (3. hafta).
- **Bakmak:** ps (-f, -ef, aux, -o), pstree, top/htop, /proc/<pid>/status (PCB).
- **STAT:** R = hazır/çalışıyor, S/D = bekliyor, T = durdurulmuş, Z = sonlandı ama toplanmadı.
- **İş denetimi:** & arka planda başlatır; Ctrl+Z durdurur; jobs, fg, bg; %n bir iş, yalın sayı bir PID'dir.
- **Sinyaller:** kill varsayılan olarak SIGTERM (15) gönderir; Ctrl+C = SIGINT (2); kill -9 = SIGKILL, son çare; $? = 128 + n.
- **Öncelik:** nice −20…19, kullanıcılar yalnızca yükseltebilir; nice 10'a karşı 0 ≈ işlemcinin 1 : 9,3'ü (5. hafta).
- **Zombiler:** onları yalnızca ebeveynin wait() çağrısı kaldırır.

GELECEK HAFTA

Kabuk Betiklerine Giriş

14. haftanın konuları:

- Neden betik: bugün yazdıklarınızı otomatikleştirin
- Shebang, chmod +x, değişkenler, tırnaklama, $( ) ve $(( ))
- $0 $1 $# $@, read, çıkış durumu $? ve exit
- test / [ ], if, for, while, fonksiyonlar, bash -x
- İki gerçek betik: ~/lab/scripts içinde backup.sh ve report.sh

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz